From Data to Solutions · Weekend 0

scikit-learn

one workflow, not an algorithm zoo

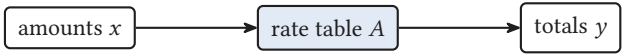
Carlos Cotrini

Section 1

From applying a rule to learning one

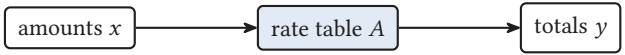
1

This morning you were handed the rule



this morning (NumPy): $y = A\,x$, forward

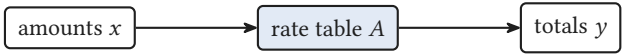
This morning you were handed the rule



this morning (NumPy): $y = A\,x$, forward

the cafe lost its price list: (shots, milk, price)

This morning you were handed the rule

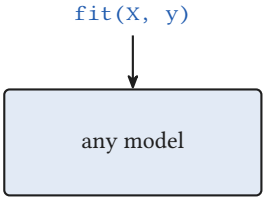


this morning (NumPy): $y = A\,x$, forward

the cafe lost its price list: (shots, milk, price)

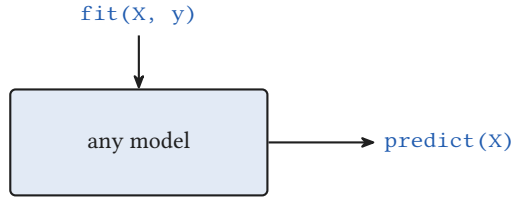
machine learning: learn the rule from examples

scikit-learn is one workflow



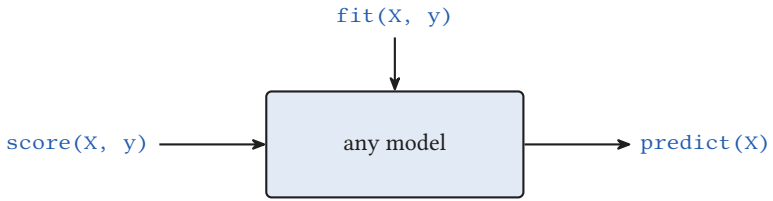
- the same three verbs for every model

scikit-learn is one workflow



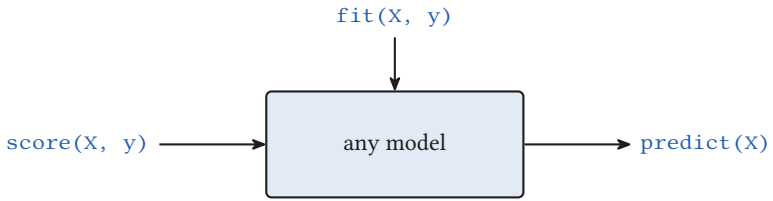
- the same three verbs for every model
- today: **two** models, not twenty

scikit-learn is one workflow



- the same three verbs for every model
- today: **two** models, not twenty
- no algorithm zoo, no tuning marathons

scikit-learn is one workflow



- the same three verbs for every model
- today: **two** models, not twenty
- no algorithm zoo, no tuning marathons

learn the workflow, not the zoo

Section 2

The estimator API

2

You could hand-roll this fit

```
for step in range(2000):
  yhat = w*shots + b
  err = yhat - price
  w = w - lr*(err*shots).mean()
  b = b - lr*err.mean()
```

gradient descent, by hand, with the NumPy you have

You could hand-roll this fit

```
for step in range(2000):
  yhat = w*shots + b
  err = yhat - price
  w = w - lr*(err*shots).mean()
  b = b - lr*err.mean()
```

```
LinearRegression().fit(X, y)
```

gradient descent, by hand, with the NumPy you have

You could hand-roll this fit

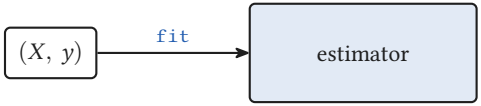
```
for step in range(2000):
  yhat = w*shots + b
  err = yhat - price
  w = w - lr*(err*shots).mean()
  b = b - lr*err.mean()
```

```
LinearRegression().fit(X, y)
```

gradient descent, by hand, with the NumPy you have

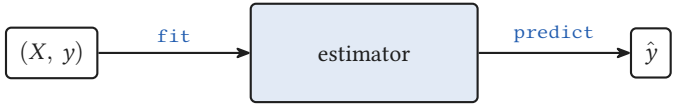
same best-fit line, one call (sklearn picks the solver)

Three verbs, one object



`fit` learn from labelled examples

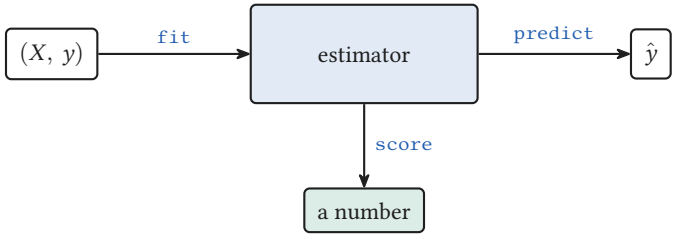
Three verbs, one object



`fit` learn from labelled examples

`predict` apply to new inputs

Three verbs, one object



`fit` learn from labelled examples

`predict` apply to new inputs

`score` grade against the truth

The machine recovers your price rule

$$\text{price} = \underbrace{2.00}_{\text{base cup}}$$

The machine recovers your price rule

$$\text{price} = 2.00 + \underbrace{0.91}_{\text{per shot}} \text{ shots}$$

The machine recovers your price rule

$$\text{price} = 2.00 + 0.91\,\text{shots} + \underbrace{0.0038}_{\text{per ml}}\,\text{milk}$$

The machine recovers your price rule

$$\text{price} = 2.00 + 0.91\,\text{shots} + \underbrace{0.0038}_{\text{per ml}}\,\text{milk}$$

true rule was $2.00 + 0.90\,\text{shots} + 0.004\,\text{milk}$

The machine recovers your price rule

$$\text{price} = 2.00 + 0.91\ \text{shots} + \underbrace{0.0038}_{\text{per ml}}\ \text{milk}$$

- learned from **180** receipts

true rule was $2.00 + 0.90$ shots $+ 0.004$ milk

The machine recovers your price rule

$$\text{price} = 2.00 + 0.91\ \text{shots} + \underbrace{0.0038}_{\text{per ml}}\ \text{milk}$$

true rule was $2.00 + 0.90$ shots $+ 0.004$ milk

- learned from **180** receipts
- `coef_` is a rate row: the A from $y = A\,x$, now learned

The machine recovers your price rule

$$\text{price} = 2.00 + 0.91\ \text{shots} + \underbrace{0.0038}_{\text{per ml}}\ \text{milk}$$

true rule was $2.00 + 0.90$ shots $+ 0.004$ milk

- learned from **180** receipts
- `coef_` is a rate row: the A from $y = A\,x$, now learned
- a flat white $(2, 150)$: CHF 4.39

The machine recovers your price rule

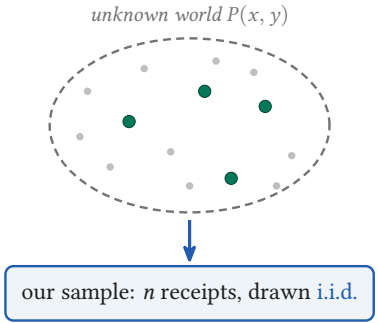
$$\text{price} = 2.00 + 0.91\ \text{shots} + \underbrace{0.0038}_{\text{per ml}}\ \text{milk}$$

true rule was $2.00 + 0.90$ shots $+ 0.004$ milk

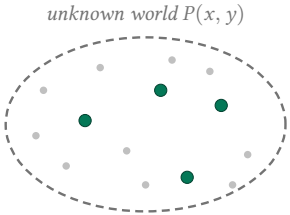
- learned from **180** receipts
- `coef_` is a rate row: the A from $y = A\,x$, now learned
- a flat white $(2, 150)$: CHF 4.39

you stopped hand-writing the optimisation

A word from learning theory

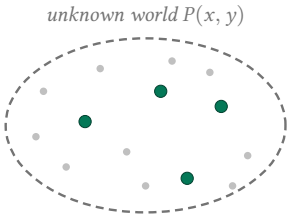


A word from learning theory



error on the sample
empirical risk: all we can measure

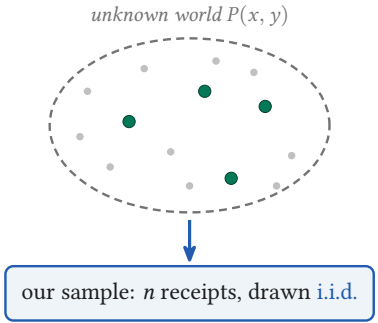
A word from learning theory



error on the sample
empirical risk: all we can measure

error on the future
true risk: what we really want

A word from learning theory

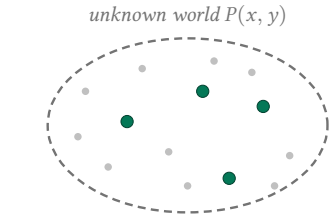


error on the sample
empirical risk: all we can measure

error on the future
true risk: what we really want

`fit`: smallest sample error over a family H
search the candidate rules (ERM)

A word from learning theory



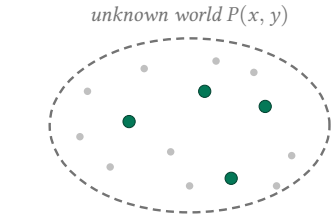
error on the sample
empirical risk: all we can measure

error on the future
true risk: what we really want

`fit`: smallest sample error over a family H
search the candidate rules (ERM)

too much capacity: aces sample, fails future
gap shrinks with more data, grows with capacity

A word from learning theory



error on the sample
empirical risk: all we can measure

error on the future
true risk: what we really want

`fit`: smallest sample error over a family H
search the candidate rules (ERM)

too much capacity: aces sample, fails future
gap shrinks with more data, grows with capacity

`fit` minimises sample error; we really want small future error

Learning theory is the three verbs

Learning theory		scikit-learn
hypothesis class H	$\mapsto$	the model, e.g. `LinearRegression()`

Learning theory is the three verbs

Learning theory		scikit-learn
hypothesis class H	$\mapsto$	the model, e.g. `LinearRegression()`
i.i.d. sample from $P(x, y)$	$\mapsto$	the data `(X, y)`, a row per example

Learning theory is the three verbs

Learning theory		scikit-learn
hypothesis class H	$\mapsto$	the model, e.g. `LinearRegression()`
i.i.d. sample from $P(x, y)$	$\mapsto$	the data `(X, y)`, a row per example
minimise empirical risk (ERM)	$\mapsto$	`model.fit(X, y)`

Learning theory is the three verbs

Learning theory		scikit-learn
hypothesis class H	$\mapsto$	the model, e.g. `LinearRegression()`
i.i.d. sample from $P(x, y)$	$\mapsto$	the data `(X, y)`, a row per example
minimise empirical risk (ERM)	$\mapsto$	`model.fit(X, y)`
apply the rule to a new input	$\mapsto$	`model.predict(X)`

Learning theory is the three verbs

Learning theory		scikit-learn
hypothesis class H	$\mapsto$	the model, e.g. `LinearRegression()`
i.i.d. sample from $P(x, y)$	$\mapsto$	the data `(X, y)`, a row per example
minimise empirical risk (ERM)	$\mapsto$	`model.fit(X, y)`
apply the rule to a new input	$\mapsto$	`model.predict(X)`
true risk: error on the future	$\mapsto$	what `score` on the test set estimates

Learning theory is the three verbs

Learning theory		scikit-learn
hypothesis class H	$\mapsto$	the model, e.g. `LinearRegression()`
i.i.d. sample from $P(x, y)$	$\mapsto$	the data `(X, y)`, a row per example
minimise empirical risk (ERM)	$\mapsto$	`model.fit(X, y)`
apply the rule to a new input	$\mapsto$	`model.predict(X)`
true risk: error on the future	$\mapsto$	what `score` on the test set estimates
capacity: how many wiggles	$\mapsto$	model complexity: degree, `max_depth`, `C`

Learning theory is the three verbs

Learning theory		scikit-learn
hypothesis class H	↦	the model, e.g. `LinearRegression()`
i.i.d. sample from $P(x, y)$	↦	the data `(X, y)`, a row per example
minimise empirical risk (ERM)	↦	`model.fit(X, y)`
apply the rule to a new input	↦	`model.predict(X)`
true risk: error on the future	↦	what `score` on the test set estimates
capacity: how many wiggles	↦	model complexity: degree, `max_depth`, `C`

`fit` minimises sample error; `score` on test estimates the future

Section 3

The cardinal rule

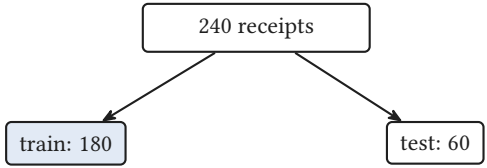
3

Never grade on what you trained on

240 receipts

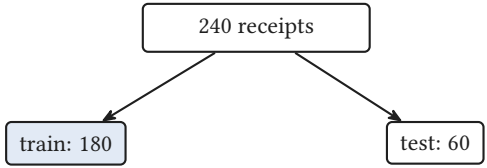
- one call: `train_test_split(X, y)`

Never grade on what you trained on



- one call: `train_test_split(X, y)`
- R^2 on train $= 0.98$, on test $= 0.98$ here

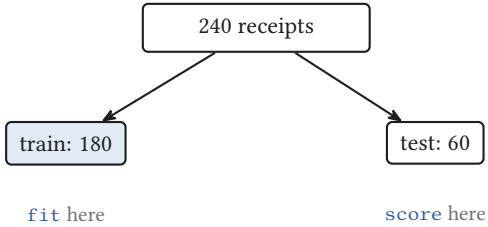
Never grade on what you trained on



`fit` here

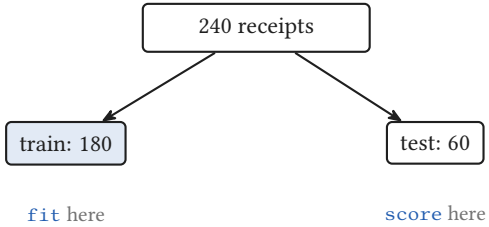
- one call: `train_test_split(X, y)`
- R^2 on train $= 0.98$, on test $= 0.98$ here
- a memoriser scores high on train, low on test

Never grade on what you trained on



- one call: `train_test_split(X, y)`
- R^2 on train $= 0.98$, on test $= 0.98$ here
- a memoriser scores high on train, low on test

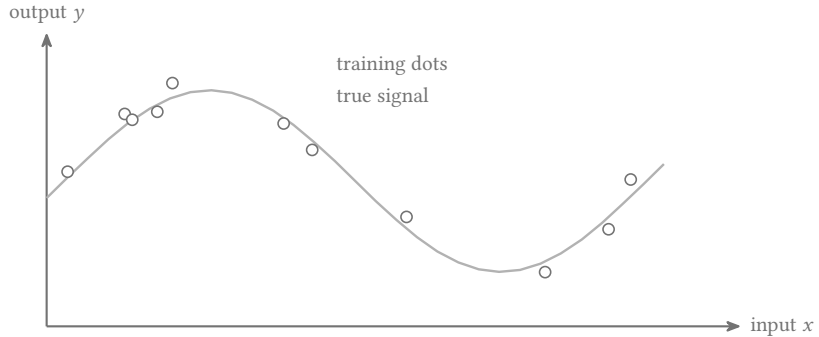
Never grade on what you trained on



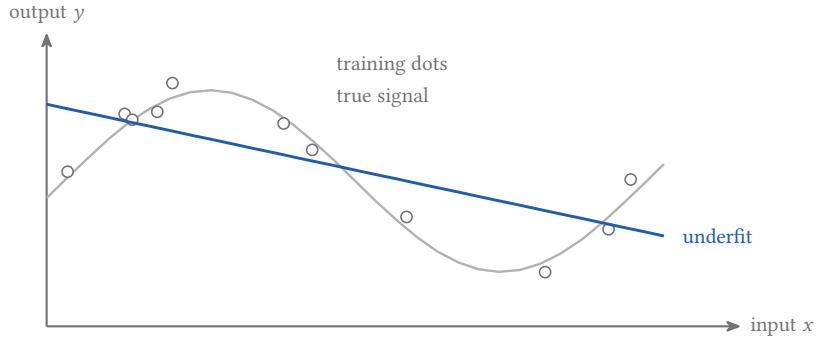
- one call: `train_test_split(X, y)`
- R^2 on train $= 0.98$, on test $= 0.98$ here
- a memoriser scores high on train, low on test

report the score on data the model never saw

Overfitting, drawn

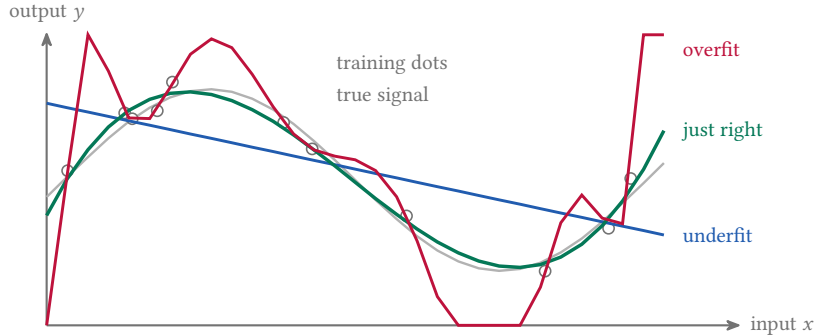


Overfitting, drawn

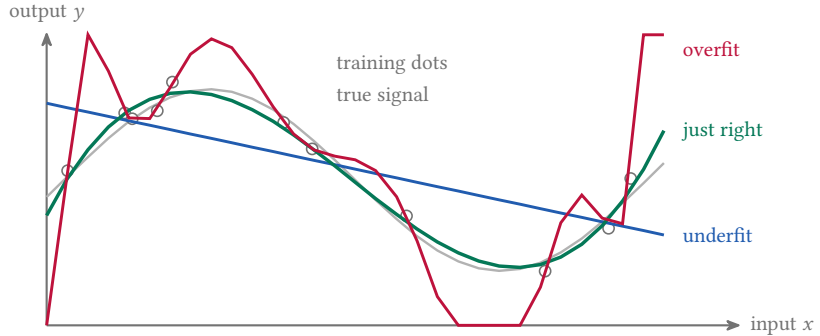


Overfitting, drawn

Overfitting, drawn

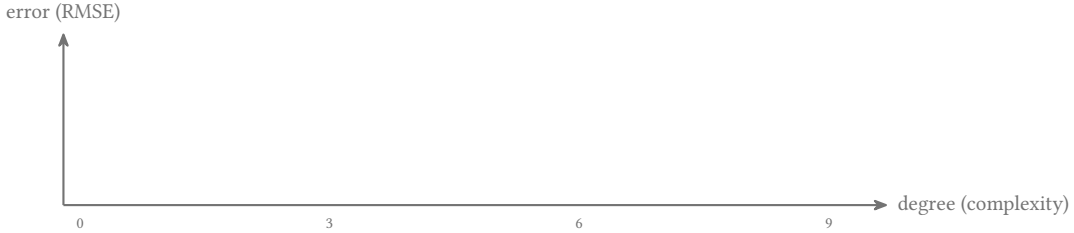


Overfitting, drawn

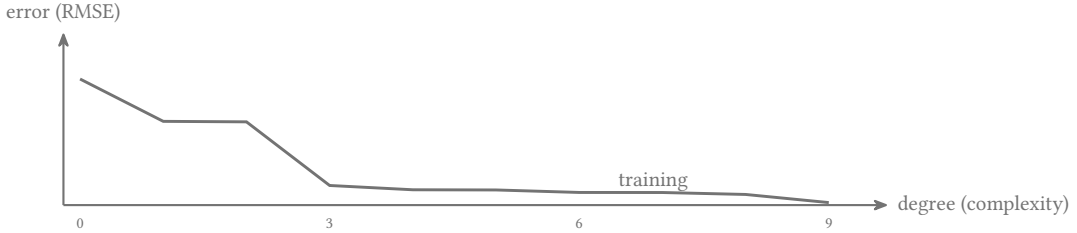


degree 9 aces the points it has seen, and fails on the next one

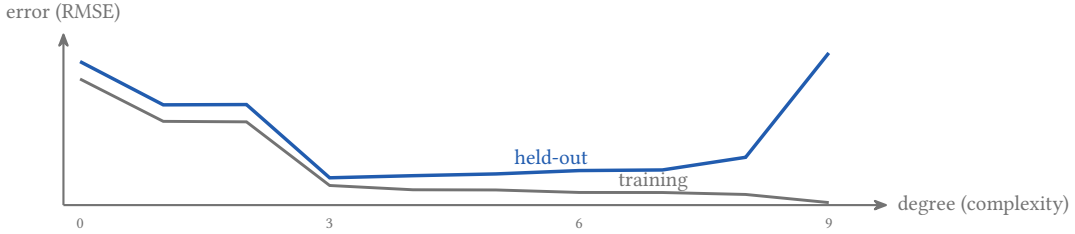
Why hold data out



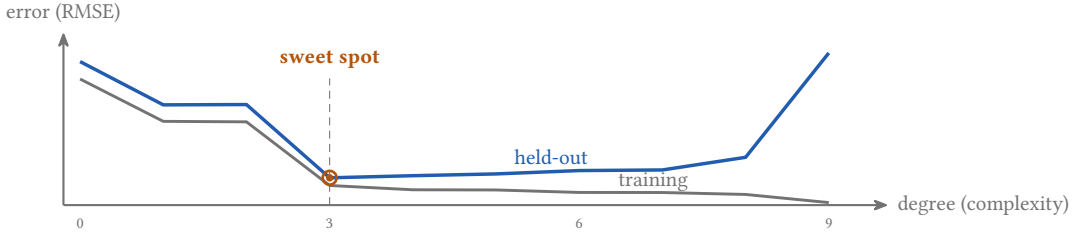
Why hold data out



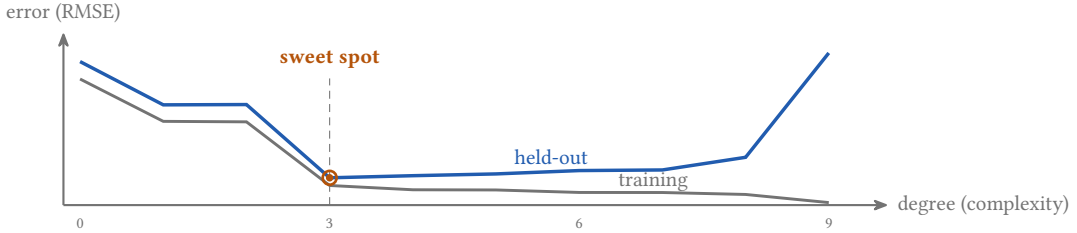
Why hold data out



Why hold data out

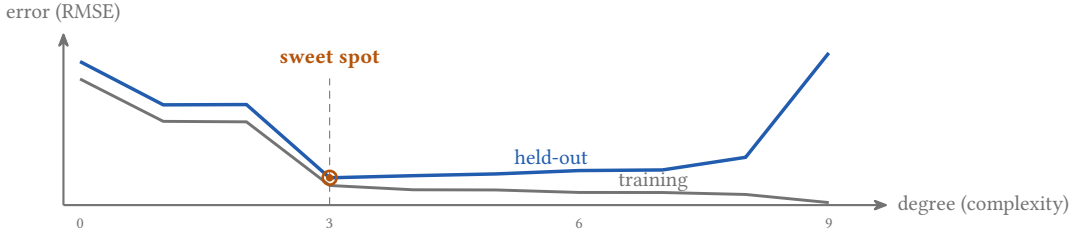


Why hold data out



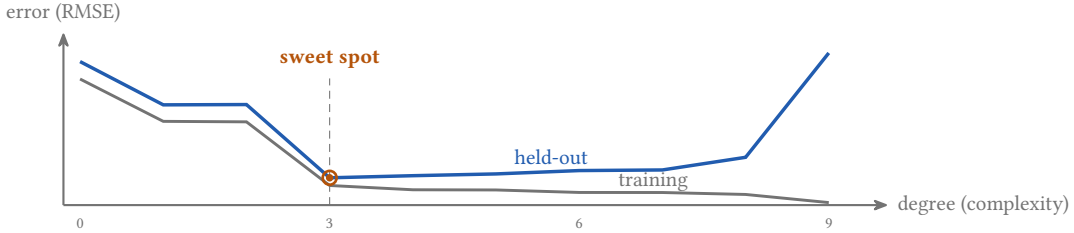
- training error always falls: blind to overfitting

Why hold data out



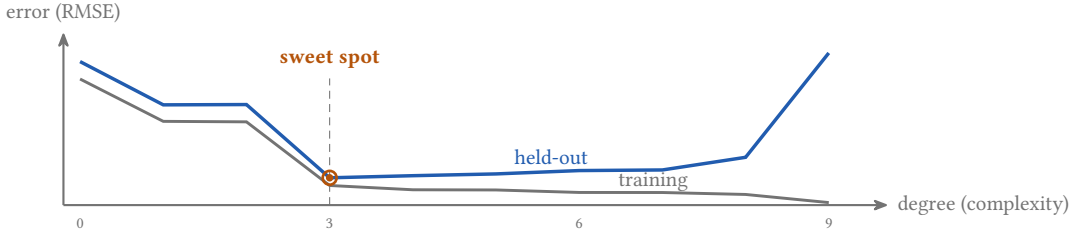
- training error always falls: blind to overfitting
- held-out error turns up: it can see it

Why hold data out



- training error always falls: blind to overfitting
- held-out error turns up: it can see it
- pick the degree at the U minimum (what `cross_val_score` estimates)

Why hold data out



- training error always falls: blind to overfitting
- held-out error turns up: it can see it
- pick the degree at the U minimum (what `cross_val_score` estimates)

training error always falls; only held-out error reveals overfitting

Section 4

A number becomes a yes/no

4

Same table, a new question

shots	milk	price	five_star
2	150	4.47	no
3	270	5.72	yes
1	0	2.88	no

- **regression**: predict a number (price)

A number becomes a yes/no

Same table, a new question

shots	milk	price	five_star
2	150	4.47	no
3	270	5.72	yes
1	0	2.88	no

- **regression**: predict a number (price)
- **classification**: predict a yes/no (a 5-star review)

A number becomes a yes/no

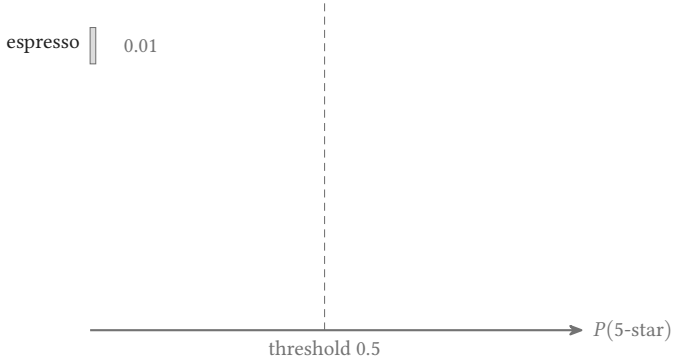
Same table, a new question

shots	milk	price	five_star
2	150	4.47	no
3	270	5.72	yes
1	0	2.88	no

- **regression**: predict a number (price)
- **classification**: predict a yes/no (a 5-star review)
- `LogisticRegression().fit(X, y)`: same three verbs

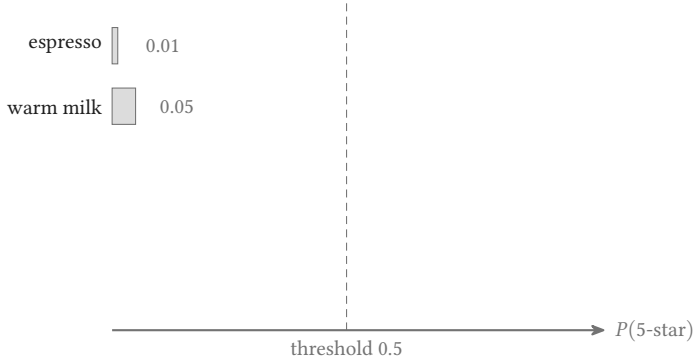
A number becomes a yes/no

A probability, not just a label



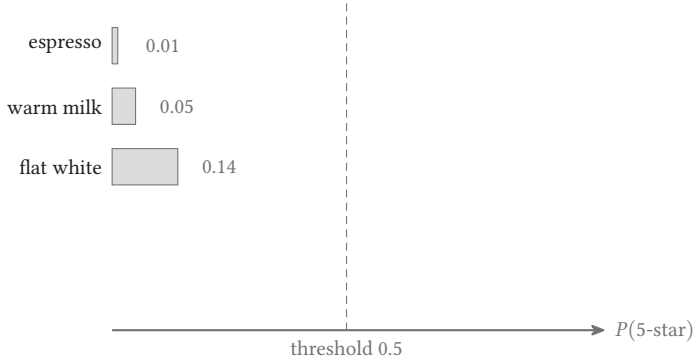
A number becomes a yes/no

A probability, not just a label

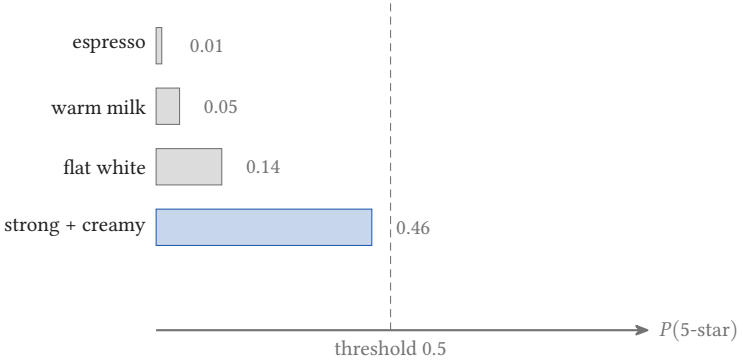


A number becomes a yes/no

A probability, not just a label

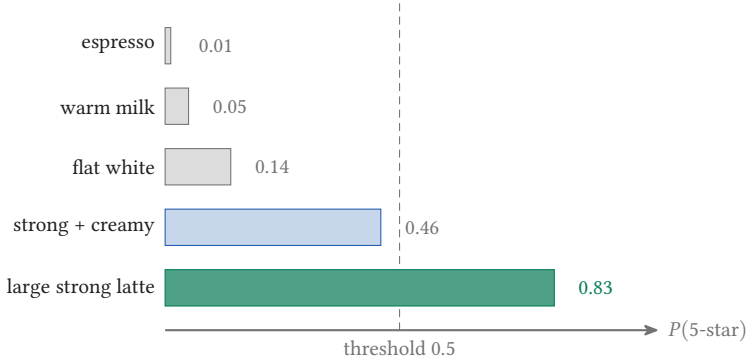


A probability, not just a label

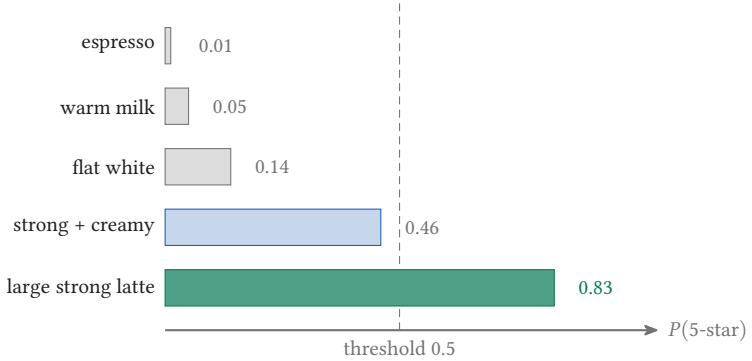


A number becomes a yes/no

A probability, not just a label

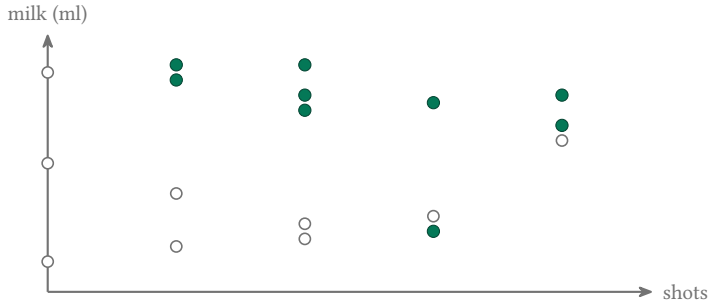


A probability, not just a label



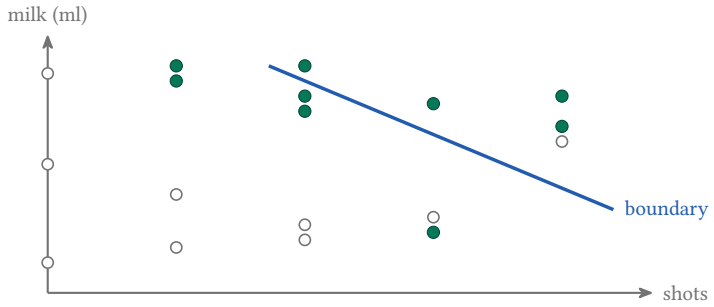
`predict_proba` grades how sure, not just yes/no

The whole model in one picture

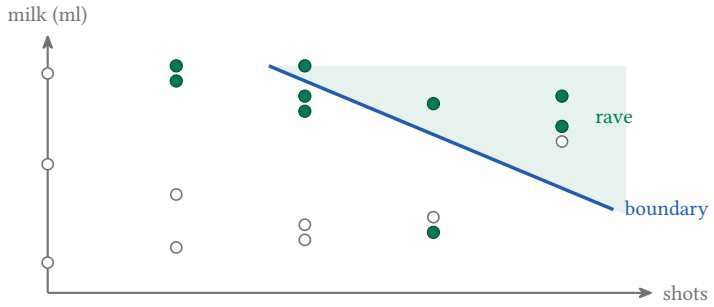


A number becomes a yes/no

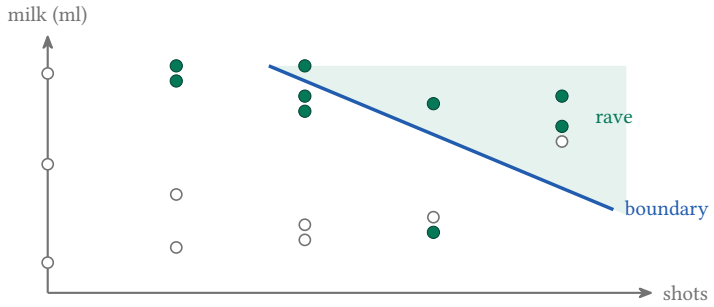
The whole model in one picture



The whole model in one picture



The whole model in one picture



logistic regression: a line, squashed to a probability

Section 5

Accuracy can lie

5

Accuracy can lie

Most drinks do not get five stars

Accuracy can lie

Most drinks do not get five stars

"always predict NO" is 73% accurate

and catches zero of the ravers

Accuracy can lie

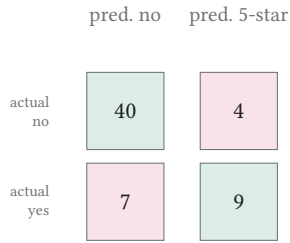
Most drinks do not get five stars

"always predict NO" is 73% accurate

and catches zero of the ravers

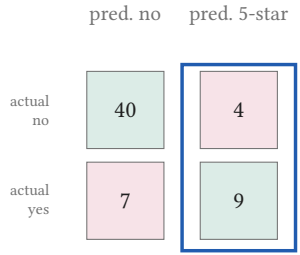
73% accurate and completely useless

Read the matrix, not the accuracy



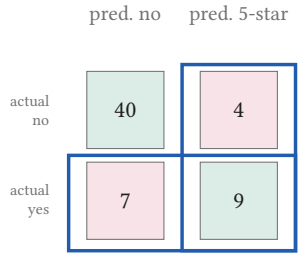
- accuracy 0.82, sounds fine

Read the matrix, not the accuracy



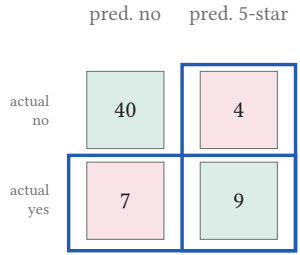
- accuracy 0.82, sounds fine
- precision 0.69: flagged raves that were real

Read the matrix, not the accuracy



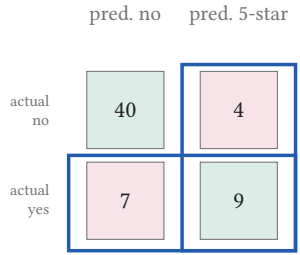
- accuracy 0.82, sounds fine
- precision 0.69: flagged raves that were real
- recall 0.56: real raves it actually caught

Read the matrix, not the accuracy



- accuracy 0.82, sounds fine
- precision 0.69: flagged raves that were real
- recall 0.56: real raves it actually caught
- ROC-AUC 0.85: one honest number

Read the matrix, not the accuracy



- accuracy 0.82, sounds fine
- precision 0.69: flagged raves that were real
- recall 0.56: real raves it actually caught
- ROC-AUC 0.85: one honest number

it misses nearly half the ravers: this is what the 2pm churn task grades

F1: one number when you need both

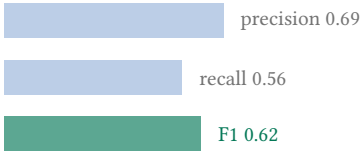
$$F_1 = \frac{2\,P\,R}{P + R}$$

F1: one number when you need both

$$F_1 = \frac{2 \times 0.69 \times 0.56}{0.69 + 0.56} = 0.62$$

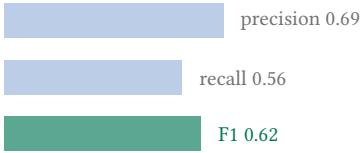
F1: one number when you need both

$$F_1 = \frac{2 \times 0.69 \times 0.56}{0.69 + 0.56} = 0.62$$



F1: one number when you need both

$$F_1 = \frac{2 \times 0.69 \times 0.56}{0.69 + 0.56} = 0.62$$



the harmonic mean: low if EITHER one is low

Section 6

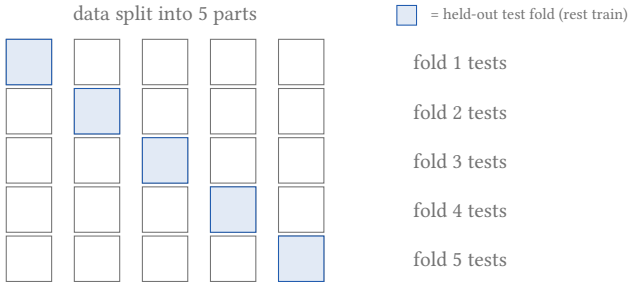
Leak-proof, and any model

6

Cross-validation: trust k splits, not one

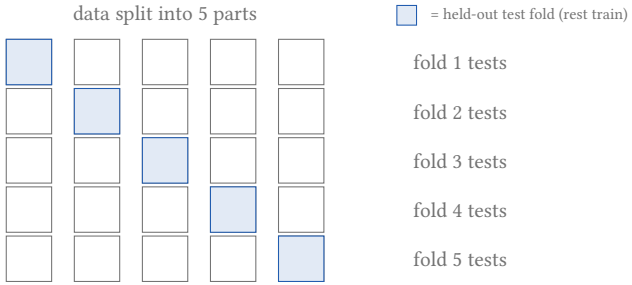
- one split scores on just 60 drinks: noisy

Cross-validation: trust k splits, not one



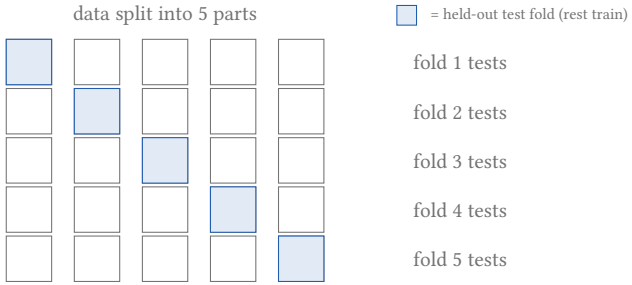
- one split scores on just 60 drinks: noisy
- 5 folds: each takes a turn as the test set, the rest train

Cross-validation: trust k splits, not one



- one split scores on just 60 drinks: noisy
- 5 folds: each takes a turn as the test set, the rest train
- `cross_val_score` averages the five scores

Cross-validation: trust k splits, not one



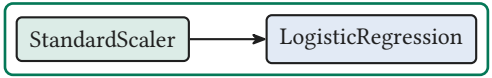
- one split scores on just 60 drinks: noisy
- 5 folds: each takes a turn as the test set, the rest train
- `cross_val_score` averages the five scores

5-fold ROC-AUC 0.85 ± 0.05, a stabler number than one split

Preprocess inside the pipeline

- scaling on the full data peeks at the test set

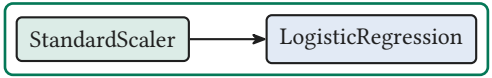
Preprocess inside the pipeline



one Pipeline, fit on the train fold only

- scaling on the full data peeks at the test set
- `make_pipeline(StandardScaler(), LogisticRegression())`

Preprocess inside the pipeline



one Pipeline, fit on the train fold only

- scaling on the full data peeks at the test set
- `make_pipeline(StandardScaler(), LogisticRegression())`
- `cross_val_score`: ROC-AUC 0.85 ± 0.05

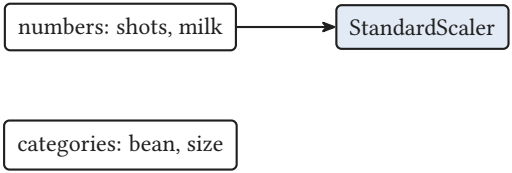
ColumnTransformer: each column type its own prep

numbers: shots, milk

categories: bean, size

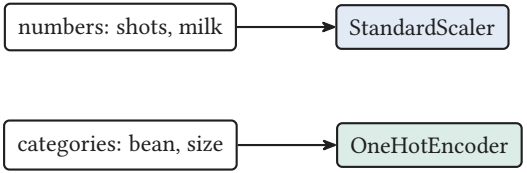
- numbers want scaling; categories want one-hot

ColumnTransformer: each column type its own prep



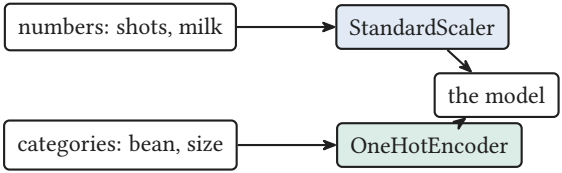
- numbers want scaling; categories want one-hot
- `ColumnTransformer` sends each column to its own step

ColumnTransformer: each column type its own prep



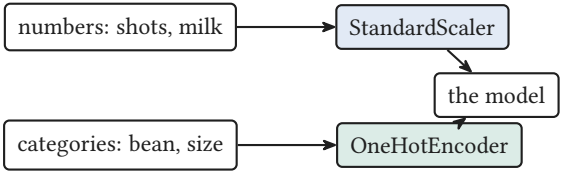
- numbers want scaling; categories want one-hot
- `ColumnTransformer` sends each column to its own step
- it drops into the same Pipeline, before the model

ColumnTransformer: each column type its own prep



- numbers want scaling; categories want one-hot
- `ColumnTransformer` sends each column to its own step
- it drops into the same Pipeline, before the model

ColumnTransformer: each column type its own prep



- numbers want scaling; categories want one-hot
- `ColumnTransformer` sends each column to its own step
- it drops into the same Pipeline, before the model

one transform per column type, then fit as usual

Leak-proof, and any model

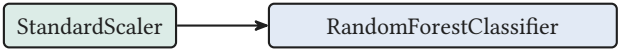
Swap one word, same workflow

`LogisticRegression()` AUC 0.85

Leak-proof, and any model

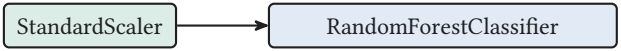
Swap one word, same workflow

`RandomForestClassifier()` AUC 0.78



Swap one word, same workflow

`RandomForestClassifier()` AUC 0.78



same `fit` · `predict` · `score` for every model (here the line wins, do not oversell the forest)

Section 7

The workflow you keep

7

The card you keep

- **split first**, before you look at anything

The card you keep

- **split first**, before you look at anything
- `fit` on train, `predict`, `score` on test

The card you keep

- **split first**, before you look at anything
- `fit` on train, `predict`, `score` on test
- the same three verbs for every model

The card you keep

- **split first**, before you look at anything
- `fit` on train, `predict`, `score` on test
- the same three verbs for every model
- imbalanced yes/no: the confusion matrix, not accuracy

The card you keep

- **split first**, before you look at anything
- `fit` on train, `predict`, `score` on test
- the same three verbs for every model
- imbalanced yes/no: the confusion matrix, not accuracy
- preprocess inside a Pipeline, so nothing leaks

The card you keep

- **split first**, before you look at anything
- `fit` on train, `predict`, `score` on test
- the same three verbs for every model
- imbalanced yes/no: the confusion matrix, not accuracy
- preprocess inside a Pipeline, so nothing leaks

scikit-learn is one workflow, and you now own it